

Test Plan

1. Introduction au plan de test système

1.1 Nom du projet de test

Système de Réservation de Vols - Plan de Test Système

1.2 Terminologie et définitions

- Réservation de Vols : Processus par lequel un passager réserve un siège sur un vol spécifique.
- Points de Fidélité : Points attribués aux passagers en fonction de la distance parcourue pendant un vol.
- Validation des Données Passagers : Vérification de la validité des informations personnelles soumises par le passager.
- Règles Métier des Vols : Ensemble des règles qui régissent la gestion des vols, y compris la disponibilité des sièges et les restrictions de passagers.
- Type de Classe : Classification des sièges dans les vols (par exemple, Première Classe, Classe Affaires, Classe Économie).

1.3 Nom de Approuver

Approuver : Flavio Castells

1.4 Objectif du Rapport :

Ce rapport présente une synthèse des tests effectués pour valider les fonctionnalités principales de l'application de réservation de vols, couvrant les scénarios de réservation, de gestion des points et des contraintes liées aux vols.

2. Description des fonctionnalités

Cette section fournit une description générale des fonctionnalités principales du **Système de Réservation de Vols** qui feront l'objet des tests. Les tests sont conçus pour valider les comportements du système en réponse aux exigences métier définies. Voici les principales fonctionnalités à tester :

2.1 Création et Validation des Réservations

Le système permet aux passagers de réserver des sièges sur un vol disponible. La fonctionnalité inclut :

- La vérification de la disponibilité des sièges selon la classe choisie (première classe, classe affaires, ou économie).
- La création de la réservation avec les données personnelles du passager (nom, numéro de passeport, âge).
- La gestion des erreurs, telles que l'absence de sièges disponibles ou la soumission de données invalides.

2.2 Attribution des Points de Fidélité

Les passagers reçoivent des points de fidélité en fonction de la distance parcourue par avion. Cette fonctionnalité inclut :

- Le calcul des points basé sur la distance du vol.
- L'accumulation des points pour un passager sur plusieurs réservations.
- L'application de règles spécifiques pour l'attribution des points en fonction des critères de vol.

2.3 Validation des Données Personnelles des Passagers

Le système valide les informations soumises par les passagers pour s'assurer de leur conformité aux exigences suivantes :

- Le nom ne peut pas être vide ni contenir de caractères invalides.
- Le numéro de passeport doit être valide (composé de chiffres et de lettres) et ne peut pas être vide.
- L'âge doit être un nombre valide, supérieur à zéro.

2.4 Gestion des Règles Métier des Vols

Le système applique plusieurs règles liées aux vols et aux passagers, telles que :

- L'existence du vol pour lequel une réservation est effectuée.
- L'unicité du passager sur un vol donné (un passager ne peut pas être ajouté deux fois sur le même vol).
- La gestion des classes de vol et de leur disponibilité.

2.5 Validation des Types de Classes

Le système valide que les passagers choisissent des types de classes valides pour leur réservation :

- Les types de classes valides sont : "First", "Business", et "Economy".
- Toute autre valeur dans le champ de type de classe doit être rejetée comme invalide.

Ces fonctionnalités sont essentielles pour garantir le bon fonctionnement de l'application de réservation de vols. Les tests associeront chaque fonctionnalité à des scénarios qui valident le comportement attendu du système, en tenant compte de la disponibilité des ressources et des règles métier.

3. Méthodologie de test système

Les tests système de l'application reposent sur une approche d'entrée-sortie à partir de requêtes JSON simulant les appels du client. Chaque scénario est conçu pour injecter des données structurées au format JSON, correspondant aux informations de réservation, de passager ou de vol. La validation du comportement du système repose ensuite sur deux critères principaux : d'une part, le **code de retour HTTP**, indiquant le succès ou l'échec de l'opération ; d'autre part, la **vérification de l'enregistrement correct des données dans un fichier CSV**, utilisé comme simulation de base de données persistante. Cette stratégie permet une évaluation cohérente de la logique métier en fin de chaîne, en imitant le plus fidèlement possible un environnement de production.

4. Fonctionnalités à tester

- Création et validation des réservations selon la disponibilité des classes.
- Attribution des points de fidélité selon les règles établies.
- Validation des données personnelles des passagers.
- Gestion des règles métier liées aux vols (existence, unicité des passagers).
- La validation des types de classe saisis, afin de s'assurer qu'ils appartiennent aux valeurs autorisées.

5. Fonctionnalités non testées

Certaines fonctionnalités ont été explicitement exclues de la portée de ces tests :

- La **logique d'annulation** et de **modification** et d', considérée hors périmètre dans cette itération. Ces fonctionnalités feront l'objet d'une phase de test distincte une fois implémentées.

6. Justification de la division des suites de tests

La division des tests en suites distinctes repose sur une approche modulaire qui favorise la clarté, la maintenabilité et la robustesse du système. Chaque suite couvre une fonctionnalité précise — qu'il s'agisse de la logique de réservation, de l'attribution des points, de la gestion des vols, ou encore de la validation des données des passagers. Cette organisation permet de localiser efficacement les erreurs et de cibler les tests à ajuster en cas d'évolution du code. Par exemple, si une nouvelle règle métier est introduite pour refuser les passagers mineurs, seuls les tests de validation des attributs devront être modifiés. De même, l'ajout d'une nouvelle classe de voyage comme « premium economy » n'affectera que la suite dédiée au type de classe, sans impacter la logique générale de réservation. Ainsi, cette structure renforce la stabilité du système logiciel tout au long de son cycle de développement.

7. Liste des cas de test

1. Test Suite : logique de la réservation

Objectif : Vérifier l'ensemble des comportements de réservation des passagers selon la classe et la disponibilité des sièges.

ID	Description	Résultat attendu	Résultat obtenu	Statut
book:suc:01	Ajouter plusieurs passagers de première classe si les sièges sont disponibles	Code 201 + données valides	Conforme	✓ Pass
book:suc:02	Ajouter un passager de chaque classe dans un vol avec toutes les classes disponibles	Code 201 + données valides	Conforme	✓ Pass
book:suc:03	Effectuer une rétrogradation de classe si la classe choisie n'est pas disponible	Code 201 + données valides	Conforme	✓ Pass
book:err:04	Ne pas ajouter un passager si aucune place n'est disponible dans aucune classe	Code 400 + données partielles valides	Non conforme	✗ Fail

2. Test Suite : Attribution des points de fidélité

Objectif : Valider l'attribution correcte des points de fidélité en fonction du vol et cumuler les points pour les réservations multiples.

ID	Description	Résultat attendu	Résultat obtenu	Statut
point:suc:01	Vérifier l'attribution correcte des points pour un passager selon la distance du vol	Code 201 + données valides	Conforme	✓ Pass
point:suc:02	Vérifier la bonne accumulation des points sur des réservations multiples	Code 201 + points cumulés	Non conforme	✗ Fail

3. Test Suite : Validation des Attributs des vols et restrictions .

Objectif : Tester la gestion des règles métier associées aux vols : existence, unicité de passager par vol, etc.

ID	Description	Résultat attendu	Résultat obtenu	Statut
flight:suc:flightNumber:01	Ajouter un passager à un vol existant	Code 201 + données valides	Conforme	✓ Pass
flight:err: flightNumber:02	Refuser l'ajout à un vol inexistant	Code 400 + aucun fichier sauvegardé	Conforme	✓ Pass
flight:err:03	Refuser l'ajout d'un même passager deux fois sur le même vol	Code 400 + un seul enregistrement	Non conforme	✗ Fail

4. Validation des Attributs de Base des Passagers

ID	Description	Résultat attendu	Résultat obtenu	Statut
TC-pass:suc:all:01	Vérifie qu'un passager avec des attributs valides est bien enregistré	Code 201 + enregistrement correct dans le fichier	Conforme	✓ Pass
TC-pass:err:name:02	Rejet si le nom contient des caractères invalides	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail
TC-pass:err:name:03	Rejet si le nom est vide	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail
TC-pass:err:passport:04	Rejet si le numéro de passeport contient des caractères invalides	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail
TC-pass:err:passport:05	Rejet si le numéro de passeport est vide	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail
TC-pass:err:age:06	Rejet si l'âge est négatif	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail
TC-pass:err:age:07	Rejet si l'âge est vide	Code 400 + aucun fichier sauvegardé	Non conforme	✗ Fail

5. Validation du Type de Classe des Passagers

ID	Description	Résultat attendu	Résultat obtenu	Statut
TC-type:suc:01	Acceptation du type de classe first avec siège disponible	Code 201 + données sauvegardées avec la bonne classe	Conforme	✓ Pass
TC-type:suc:02	Acceptation du type de classe business avec siège disponible	Code 201 + données sauvegardées avec la bonne classe	Conforme	✓ Pass
TC-type:suc:03	Acceptation du type de classe economy avec siège disponible	Code 201 + données sauvegardées avec la bonne classe	Conforme	✓ Pass
TC-type:err:04	Rejet du type de classe default (non reconnu)	Code 400 + aucune sauvegarde	Conforme	✓ Pass

8 . Rendement de la conception des cas de test

$$\text{TDCY} = \frac{\text{NPT}}{\text{NPT} + \text{Number of TCE}} \times 100\%$$

$$\text{TDCY} = (21 / 21 + 9) * 100$$

$$\text{TDCY} = 70\%$$